SentinelAI: ผู้ช่วยคัดกรองเหตุการณ์ความปลอดภัยอัจฉริยะสำหรับ SOC

รายงานโครงงานรายวิชาการพัฒนาแอปพลิเคชันด้วย LLM (ฉบับภาษาไทย)

โมเดล gemma-4-E4B-it ผ่าน OpenAI-compatible proxy ของรายวิชา · ผล retrieval สร้างเมื่อ 2026-10-16 · ผล end-to-end ล่าสุด 2026-10-17 · ชุดข้อมูล eval/datasets/retrieval_eval_v1.jsonl (sha256/16 ef03063861a8e6be)

บทคัดย่อ

SentinelAI ช่วยนักวิเคราะห์ Security Operations Center (SOC) ระดับเริ่มต้นคัดกรอง alert ความปลอดภัยแบบสังเคราะห์ โดยรวม structured extraction ที่ตรวจด้วย schema และการตรวจ grounding แบบตรงตัวอักษร, retrieval-augmented generation บน playbook สมมติที่เขียนขึ้นเอง 19 ฉบับ, agent ที่เรียกใช้เครื่องมืออ่านอย่างเดียว 3 ตัวภายใต้ขอบเขตที่กำหนด, ตารางคำนวณความรุนแรงแบบกำหนดแน่นอน และรายงานเหตุการณ์ที่ตรวจการอ้างอิงแล้ว ให้บริการผ่าน FastAPI และ web dashboard บนชุด test ที่แยกไว้ 16 คำถาม (ตอบได้ 14 ข้อ) ค่าที่เลือกจากชุด dev (hybrid-sbert-section-a0.3) เพิ่ม Recall@3 จาก 0.679 เป็น 0.750 และ MRR@5 จาก 0.696 เป็น 0.729 เทียบกับ baseline BM25 แบบ chunk คงที่ ซึ่งเป็นการปรับปรุงเล็กน้อยบนชุดทดสอบขนาดเล็ก สำหรับการประเมินแบบ end-to-end กับ Gemma จริง ระบบถูกปรับปรุง 3 รอบ โดยแต่ละรอบวัดผลบนชุด holdout ใหม่ที่เขียนไว้ก่อนแก้ (หัวข้อ 7)

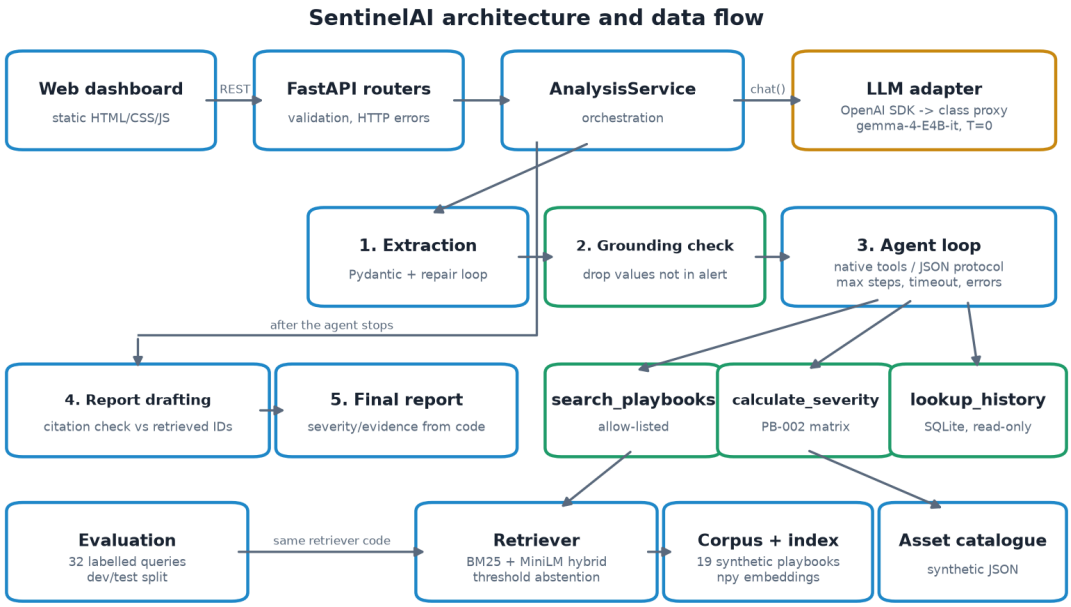
1. ที่มาและปัญหา

นักวิเคราะห์ระดับ Tier-1 ต้องรับมือกับ alert จำนวนมากในหลายรูปแบบ ข้อผิดพลาดที่พบบ่อยของผู้เริ่มต้นคือ (ก) สับสน alert ที่ดูคล้ายกันแต่ต้องใช้ playbook ต่างกัน เช่น การเดารหัสผ่านจำนวนมากกับการ login สำเร็จหลังจากนั้น (ข) เติมข้อมูลที่ขาดด้วยการเดา และ (ค) ให้ระดับความรุนแรงไม่สม่ำเสมอ เครื่องมือแชตกับเอกสารทั่วไปมักทำให้ข้อ (ข) และ (ค) แย่ลง เพราะโมเดลภาษาสามารถสร้างชื่อโฮสต์ การอ้างอิง และลำดับความสำคัญขึ้นมาเองอย่างมั่นใจ SentinelAI จึงออกแบบตามหลักตรงข้าม: ให้โมเดลจัดการเรื่องภาษา ส่วนทุกอย่างที่ต้องทำซ้ำได้หรือตรวจสอบได้ให้โค้ดเป็นผู้ทำและตรวจเทียบกับหลักฐาน

2. กลุ่มผู้ใช้และวัตถุประสงค์

- **ผู้ใช้:** นักวิเคราะห์ SOC ระดับเริ่มต้นและนักศึกษาที่เรียนการคัดกรองเหตุการณ์ รวมถึงผู้สอนที่ตรวจขั้นตอนการทำงาน
- **O1** ดึงข้อมูลเหตุการณ์แบบมีโครงสร้างโดยไม่สร้างค่าขึ้นเอง (Pydantic + การตรวจ grounding)
- **O2** ค้นหา playbook ที่ถูกต้องและอ้างอิงเฉพาะเอกสารที่ค้นได้จริง (RAG + การตรวจการอ้างอิง)
- **O3** ให้ agent เลือกเครื่องมือสืบสวนได้อย่างปลอดภัย (allow-list, schema, เงื่อนไขหยุดที่ชัดเจน)
- **O4** คำนวณความรุนแรงแบบกำหนดแน่นอนจากนโยบายที่มีเอกสาร (PB-002) และแค็ตตาล็อกทรัพย์สิน
- **O5** วัดคุณภาพ retrieval เทียบ baseline ด้วยโปรโตคอล dev/test และรายงานตามจริง

3. สถาปัตยกรรมระบบและการไหลของข้อมูล



ภาพที่ 1 ชั้นของระบบและลำดับการทำงานของคำขอ กล่องสีเขียวคือส่วนประกอบด้านความปลอดภัยที่ทำงานแบบกำหนดแน่นอน

Router ทำหน้าที่เพียงตรวจ input ของ HTTP และแปลงข้อผิดพลาด (422 input ไม่ถูกต้อง, 400 รหัสเอกสารผิดรูปแบบ, 502 ผลลัพธ์โมเดลไม่ผ่านการตรวจ, 503 ยังไม่ได้ตั้งค่า proxy) *AnalysisService* ควบคุมลำดับงานทั้งหมด ได้แก่ extraction, grounding, agent loop, การบังคับเรียกเครื่องมือ, การร่างรายงาน และการบันทึกผล Repository ห่อ SQLite (ผลวิเคราะห์ที่บันทึกและประวัติเหตุการณ์สังเคราะห์ ใช้ parameterised SQL เท่านั้น) และแค็ตตาล็อกทรัพย์สิน JSON มี LLM adapter เพียงตัวเดียวที่สื่อสารกับโมเดล ทำให้ทดสอบทั้ง pipeline ได้ด้วย client ปลอมที่กำหนดคำตอบไว้ล่วงหน้า

4. การเชื่อมต่อ LLM, RAG, Structured Output และการออกแบบ Agent

การเชื่อมต่อ LLM ใช้ OpenAI Python SDK ทางการ ชี้ไปที่ proxy ของรายวิชาด้วย OPENAI_API_KEY และ OPENAI_BASE_URL จาก environment ใช้โมเดล gemma-4-E4B-it ที่ temperature 0 ขอ JSON mode และตัดออกอัตโนมัติหาก proxy ไม่รองรับ ผลการตรวจความเข้ากันได้ของ proxy: list_models=ผ่าน, chat=ผ่าน, system_role=ผ่าน, json_mode=ผ่าน, native_tools=ผ่าน

Structured output ผลการดึงข้อมูลและร่างรายงานถูกตรวจด้วย Pydantic (ประเภทเหตุการณ์แบบ enum, ตรวจ IP, จำกัดขนาด list) หากไม่ผ่านจะส่งข้อความผิดพลาดกลับให้โมเดลแก้ สูงสุด 3 ครั้ง ถ้ายังไม่ผ่าน API จะตอบ 502 พร้อมระบุขั้นตอนที่ล้มเหลว แทนการสร้างรายงานปลอม หลังผ่านการตรวจ ค่าทุกค่าที่ดึงได้ (ชื่อโฮสต์, IP, ผู้ใช้, เวลา, หลักฐาน, indicator) ต้องปรากฏตรงตัวใน alert มิฉะนั้นจะถูกลบและระบุเป็นข้อจำกัด ความน่าเชื่อถือของ alert ไม่ได้ให้โมเดลเลือกระดับเองโดยตรง แต่ให้จัดประเภทหลักฐาน (specific telemetry / anomaly / vague) แล้วโค้ดแปลงเป็นระดับ confidence แบบกำหนดแน่นอน (ดูหัวข้อ 7)

RAG playbook แต่ละฉบับมีรหัสคงที่ (PB-001..PB-019) ชื่อ เวอร์ชัน หมวดหมู่ และแหล่งที่มา และระบุชัดว่าเป็นเอกสารสมมติ เอกสารถูกทำความสะอาด (ไม่ index ข้อความประกาศที่ซ้ำกัน) และแบ่งตามหัวข้อ markdown โดยเติมชื่อเอกสารและหัวข้อไว้ด้านหน้า การค้นหาให้คะแนน chunk ด้วย BM25 และ cosine similarity ของ all-MiniLM-L6-v2 แล้วรวมเป็น alpha·cos + (1−alpha)·bm25/(bm25+8) จัดอันดับเอกสารตาม chunk ที่ดีที่สุด และงดตอบเมื่อคะแนนสูงสุดต่ำกว่าเกณฑ์ที่ปรับจากชุด dev รายงานอ้างอิงได้เฉพาะรหัสที่ค้นได้จริง หากผิดจะสั่งแก้ และหากยังผิดจะคืนข้อผิดพลาด

Agent มีเครื่องมือ search_playbooks, calculate_severity และ lookup_incident_history ทั้งหมดอ่านอย่างเดียว แต่ละตัวมี Pydantic argument model (ห้าม field เกิน, จำกัดค่าจำนวนเต็ม, regex ชื่อโฮสต์) ใช้ native OpenAI tool calling เมื่อรองรับ มิฉะนั้นใช้ JSON action protocol ที่มีเอกสารและระบุไว้ใน trace loop หยุดเมื่อได้คำตอบสุดท้าย, ครบ 6 ครั้ง, เกิน 150 วินาที, ผิดพลาดติดกัน 3 ครั้ง หรือการเชื่อมต่อล้มเหลว และปฏิเสธการเรียกซ้ำ หาก agent ข้ามการค้นหา ค้นแล้วไม่พบอะไร หรือคำนวณความรุนแรงด้วยปัจจัยที่ต่างจาก extraction orchestrator จะเรียกเครื่องมือเองและบันทึกเป็นขั้น enforced ข้อความ alert, ข้อความจาก playbook และผลจากเครื่องมือถูกครอบด้วย data tag และถือเป็นข้อมูลที่ไม่น่าเชื่อถือ

ความรุนแรง score = tier(1–4) + confidence(1–3) + 2×impact(0–3) + 2 หากเป็นบัญชีสิทธิ์สูง ช่วงคะแนน 0–4 Informational, 5–6 Low, 7–8 Medium, 9–10 High, 11 ขึ้นไป Critical ค่า tier มาจากแค็ตตาล็อกเสมอ ไม่ได้มาจากโมเดล ทรัพย์สินที่ไม่รู้จักใช้ค่า medium และถูกแจ้งให้ตรวจ

5. การสร้างชุดข้อมูลและวิธีการ

ชุดประเมิน retrieval มีคำถามที่เขียนเอง 32 ข้อ (dev 16, test 16) แต่ละข้อมีรหัสเอกสารที่เกี่ยวข้อง ประเภทเหตุการณ์ที่คาดหวัง ระดับความรุนแรงที่คาดหวังพร้อมปัจจัย (เมื่อระบุได้) การงดตอบที่คาดหวัง และหมายเหตุอธิบาย label สัดส่วนสถานการณ์: ทำทวม 1, ตอบได้ 17, false positive 2, หลักฐานไม่พอ 2, นอกขอบเขต 4, คำถามนโยบาย 3, คล้ายแต่คนละ playbook 3 การตรวจอัตโนมัติยืนยันว่ารหัสเอกสารมีอยู่จริง, label ความรุนแรงตรงกับสูตร PB-002 และไม่มีคำถามใดใช้ข้อความ 6 คำติดกันซ้ำกับ corpus สำหรับการประเมิน end-to-end มีชุด holdout เพิ่มอีก 3 ชุด (v2, v3, v4 ชุดละ 16 alert) ซึ่งเขียนและใส่ label ก่อนการปรับปรุงแต่ละรอบ ข้อมูลทั้งหมดเป็นข้อมูลสังเคราะห์ IP มาจากช่วงสำหรับเอกสาร RFC 5737

6. ตัวชี้วัดและ baseline

- Recall@3 = จำนวนเอกสารเกี่ยวข้องใน 3 อันดับแรก / จำนวนเอกสารเกี่ยวข้อง; MRR@5 = 1/อันดับของเอกสารเกี่ยวข้องตัวแรกใน 5 อันดับแรก (ถ้าไม่มีเป็น 0); Precision@3 และ Hit@1 เป็นตัวชี้วัดรอง ทั้งหมดเฉลี่ยเฉพาะคำถามที่**ตอบได้**
- ความถูกต้องของการงดตอบคิดจาก**ทุก**คำถาม: ถูกเมื่อระบบงดตอบในกรณีที่ไม่มีเอกสารเกี่ยวข้องเท่านั้น
- **Baseline A**: BM25 บน chunk คงที่ 120 คำ ซึ่งเป็นการทำแบบตรงไปตรงมาครั้งแรก
- **ตัวเลือกที่ทดลอง**: BM25 แบ่งตามหัวข้อ; dense LSA (TF-IDF + SVD, seed 42); dense MiniLM; hybrid ที่ alpha ∈ {0.3, 0.5, 0.7}
- **โปรโตคอล**: ปรับเกณฑ์บน dev เท่านั้น และกำหนดกฎการเลือกไว้ล่วงหน้า: max dev mean(Recall@3, MRR@5); tie-break dev abstention accuracy; then list order รายงานผล test ครั้งเดียวสำหรับ baseline และค่าที่เลือก (ค่าอื่นแสดงเพื่อความโปร่งใสแต่ไม่ได้ใช้ตัดสินใจ)
- **End-to-end**: ผลลัพธ์ผ่าน schema, ประเภทเหตุการณ์ถูก, ความรุนแรงถูก, อ้างอิงเฉพาะเอกสารที่ค้นได้, อ้างอิง playbook ที่เกี่ยวข้อง และการส่งเคส undetermined ให้คนตรวจ

7. ผลการทดลอง

Retrieval

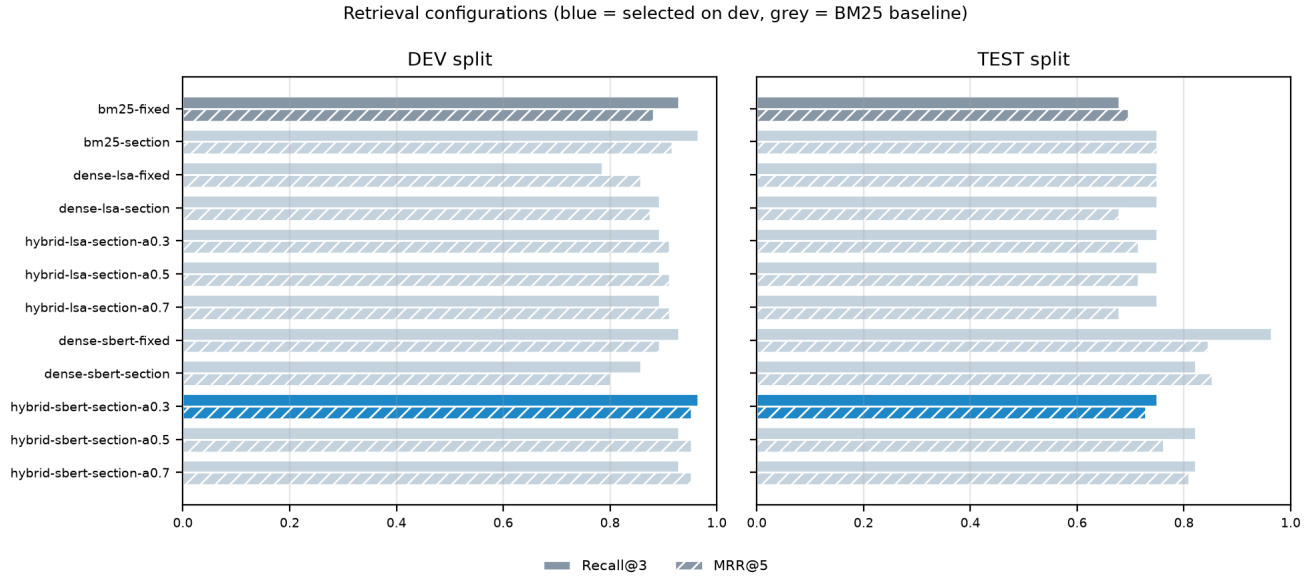
ตารางที่ 1 baseline เทียบกับค่าที่เลือกบนชุด test (16 คำถาม, ตอบได้ 14 ข้อ)

ชุดทดสอบ (test)	Recall@3	MRR@5	P@3	Hit@1	งดตอบถูก
A: bm25-fixed	0.679	0.696	0.238	0.643	0.875
B: hybrid-sbert-section-a0.3 (เลือกจาก dev)	0.750	0.729	0.262	0.643	0.875

เทียบ reciprocal rank รายคำถามบน test กับ baseline: ชนะ 2 แพ้ 1 เสมอ 11 บนชุด dev ค่าที่เลือกได้ Recall@3 0.964 / MRR@5 0.952 เทียบกับ 0.929 / 0.881 ของ baseline

ตารางที่ 2 ทุก configuration (การเลือกใช้เฉพาะคอลัมน์ dev)

Configuration	chunks	dev R@3	dev MRR@5	dev งดตอบ	test R@3	test MRR@5	test งดตอบ
bm25-fixed	40	0.929	0.881	1.000	0.679	0.696	0.875
bm25-section	85	0.964	0.917	1.000	0.750	0.750	0.875
dense-lsa-fixed	40	0.786	0.857	0.875	0.750	0.750	0.875
dense-lsa-section	85	0.893	0.875	0.875	0.750	0.679	0.875
hybrid-lsa-section-a0.3	85	0.893	0.911	1.000	0.750	0.714	0.812
hybrid-lsa-section-a0.5	85	0.893	0.911	0.875	0.750	0.714	0.875
hybrid-lsa-section-a0.7	85	0.893	0.911	0.875	0.750	0.679	0.875
dense-sbert-fixed	40	0.929	0.893	1.000	0.964	0.845	1.000
dense-sbert-section	85	0.857	0.800	1.000	0.821	0.854	1.000
hybrid-sbert-section-a0.3	85	0.964	0.952	1.000	0.750	0.729	0.875
hybrid-sbert-section-a0.5	85	0.929	0.952	1.000	0.821	0.762	0.938
hybrid-sbert-section-a0.7	85	0.929	0.952	1.000	0.821	0.809	1.000



ภาพที่ 2 Recall@3 และ MRR@5 ของแต่ละ configuration บน dev และ test (สร้างจาก eval/results/retrieval_runs.json)

การประเมิน end-to-end กับ Gemma

ทุก alert ที่มี label ถูกส่งผ่าน pipeline จริงทั้งหมด การรันครั้งแรก (r0) พบว่า Gemma ให้ alert confidence เป็น “medium” เกือบทุกเคส และประเมิน impact ต่ำกว่าความจริง จึงปรับปรุง 3 รอบ (บันทึกใน docs/EXPERIMENT_LOG.md) ก่อนแต่ละรอบได้เขียนชุด holdout ใหม่ 16 alert พร้อม label ทำให้ทุกการเปลี่ยนแปลงมีการวัดผลบนข้อมูลที่ไม่ได้ใช้ออกแบบอย่างน้อยหนึ่งชุด (ตารางที่ 4) ส่วนชุด test เดิมถูกดูเคสที่ผิดหลัง r0 แล้ว จึงไม่ใช่การวัดแบบ blind อีกต่อไป

ตารางที่ 4 แต่ละรอบวัดผลบนชุด holdout ที่เขียนไว้ก่อนรอบนั้น (ก่อน → หลัง)

รอบ	สิ่งที่เปลี่ยน	ชุดที่ไม่เคยเห็น	Severity	อ้างอิงที่เกี่ยวข้อง
r1	นิยาม confidence/impact ชัดเจนใน prompt; ค้นหาคู่มือหลักฐานไม่พอเมื่อ undetermined	holdout v2 (16)	0.462 → 0.615	0.750 → 0.812
r2	จัดประเภทหลักฐาน → confidence แบบกำหนดแน่นอน; ค้นซ้ำเมื่อ agent ค้นไม่พบ	holdout v3 (16)	0.923 → 0.923	0.812 → 0.938
r3	คำถามใช่/ไม่ใช่ “ผลการโจมตียืนยันแล้วหรือไม่”; ค้นด้วยชื่อประเภทเป็นทางสุดท้าย	holdout v4 (16)	0.615 → 0.846	1.000 → 1.000

ตารางที่ 5 ชุด test (12 alert, มี label ความรุนแรง 7 ข้อ) รันครั้งแรกเทียบกับฉบับสุดท้าย (ไม่ใช่ blind หลัง r0)

ตัวชี้วัด (ชุด test)	r0	ฉบับสุดท้าย
ผลลัพธ์ผ่าน schema	1.000	1.000
ประเภทเหตุการณ์ถูก	1.000	1.000
ระดับความรุนแรงถูก	0.571	1.000
อ้างอิงเฉพาะเอกสารที่ค้นได้	1.000	1.000
อ้างอิง playbook ที่เกี่ยวข้อง	0.917	1.000
เคส undetermined ส่งให้คนตรวจ	1.000	1.000
ใช้ native tool calling	1.000	1.000

รันครั้งสุดท้าย 2026-10-17T07:27:11+00:00 (r3_outcome_flag_label_fallback) การรัน r2 ซ้ำได้ระดับความรุนแรงและประเภทเหตุการณ์เหมือนเดิมทุกเคส ความต่างระหว่างรอบจึงไม่ใช่ความแปรปรวนจากการสุ่ม r3 พลาดเพิ่ม 1 เคสความรุนแรงและ 1 การอ้างอิงบน holdout v3 (ชุดที่เคยเห็นแล้ว) แต่เมื่อรวมทุกชุด จำนวนความรุนแรงที่ถูกเพิ่มจาก 40/51 เป็น 43/51

8. กรณีที่สำเร็จและการวิเคราะห์ความผิดพลาด

กรณีที่สำเร็จ การแบ่ง chunk ตามหัวข้อเพียงอย่างเดียวเพิ่ม Recall@3 ของ BM25 บน test จาก 0.679 เป็น 0.750 เพราะชื่อหัวข้ออย่าง “Recognising the pattern” และ “Indicators” รวมคำที่แยกแยะได้ไว้ด้วยกัน hybrid ดึงขั้นตอนสำหรับหลักฐานไม่พอของ alert ที่คลุมเครือ (Q24) ได้ ซึ่ง baseline จัดไว้อันดับที่สี่ การทดสอบ pipeline ด้วย LLM จำลองแสดงว่าการอ้างอิงที่แต่งขึ้นถูกแก้หรือปฏิเสธ ชื่อโฮสต์ที่แต่งขึ้นถูกลบ และคำขอ “run_shell” ที่ถูกแทรกมาถูกปฏิเสธ

ตารางที่ 3 ความผิดพลาดที่เหลือของ retrieval configuration ที่เลือกบนชุด test

คำถาม	สถานการณ์	เอกสารที่ถูก	Top-5 ที่ได้	สาเหตุ (วิเคราะห์เอง)
Q02	ตอบได้	PB-004	PB-018, PB-013, PB-005, PB-006, PB-004	ชื่อโฮสต์ LAB-TEST-07 ไปตรงกับแค็ตตาล็อกทรัพย์สิน (PB-018) จึงแซง PB-004
Q04	ตอบได้	PB-005	PB-016, PB-001, PB-018, PB-004, PB-015	ไม่มีคำซ้ำกัน (ชื่อเมือง) และคะแนน dense ต่ำ จึงงดตอบผิดพลาด
Q20	คล้ายแต่คนละ playbook	PB-017|PB-005	PB-017, PB-004, PB-012, PB-013, PB-003	เอกสารเกี่ยวข้องตัวที่สอง (PB-005) อยู่ต่ำกว่าอันดับ 3
Q26	คำถามนโยบาย	PB-016	PB-001, PB-018, PB-005, PB-011, PB-006	คำศัพท์ไม่ตรงกัน: “cut off the network” กับ “isolate”
Q30	นอกขอบเขต	ไม่มี (ควรงดตอบ)	PB-005, PB-015, PB-017, PB-018, PB-001	คำทั่วไปทางธุรกิจได้คะแนนสูงกว่าเกณฑ์งดตอบเล็กน้อย

ข้อสังเกตตามจริง (1) ตัวชี้วัดลดลงจาก dev ไป test ทุก configuration แสดงว่าชุด dev ง่ายกว่า และด้วยคำถามที่ตอบได้เพียง 14 ข้อ คำถามเดียวเปลี่ยน Recall@3 ได้ประมาณ 0.07 (2) dense-sbert-fixed ดีที่สุดบน test แต่ไม่ได้ถูกเลือก เพราะการเลือกตอนนี้เท่ากับการจูนกับชุด test (3) end-to-end ที่ยังพลาด: impact ของการโจมตีที่ยืนยันแล้วบางเคสยังเป็น “suspected” และการจัดประเภทหลักฐานไม่คงที่ในเคสท้ำกึ่ง (4) เคสที่ตั้งใจให้กำกวมบางเคสถูกจัดเป็น undetermined ซึ่งปลอดภัยเพราะถูกส่งให้คนตรวจเสมอ

9. ความปลอดภัย

- ไม่มีเครื่องมือ shell, ระบบไฟล์, การสแกนเครือข่าย หรือการเขียนข้อมูล ชื่อเครื่องมือต้องอยู่ใน allow-list และ argument ถูกตรวจด้วย schema (มีการทดสอบ path traversal และ SQL injection)
- Prompt injection: เนื้อหาที่ไม่น่าเชื่อถือถูกระบุเป็นข้อมูล วลีที่เป็นการแทรกคำสั่งถูกตรวจจับและบังคับให้คนตรวจ และข้อจำกัดเชิงสถาปัตยกรรมทำให้การใช้เครื่องมือผิดวัตถุประสงค์ไม่เกิดผลเสีย
- การอ้างอิงต้องเป็นรหัสที่ค้นได้จริง ความรุนแรงคำนวณด้วยโค้ด และค่าที่ไม่มีในหลักฐานถูกลบ
- Secret มาจาก environment variable เท่านั้น (.env ถูก git-ignore) ไม่ถูก log หรือส่งกลับ ข้อความผิดพลาดถูก redact จำกัดขนาด request 64 KiB (413) และวิเคราะห์พร้อมกันได้สูงสุด 2 งาน (429) ใช้ CSP เข้มงวดและแสดงผลด้วย textContent

10. ข้อจำกัดและงานในอนาคต

- corpus และชุดข้อมูลเป็นข้อมูลสังเคราะห์ขนาดเล็ก ผลลัพธ์ไม่สามารถใช้แทนข้อมูล SOC จริงได้
- ชุด end-to-end มีขนาดเล็ก (12–16 alert) หนึ่งเคสเปลี่ยนความแม่นยำได้ 6–20 จุด
- grounding แบบตรงตัวอักษรตัดค่าที่ถูกแต่เขียนต่างออกไป และการตรวจ injection แบบ pattern อาจถูกเลี่ยงได้
- ใช้เกณฑ์งดตอบค่าเดียว ไม่มีระบบยืนยันตัวตนหรือแยกผู้ใช้หลายคน จึงควรรันในเครื่องเท่านั้น
- อนาคต: เชื่อมโยงหลาย alert, รับ feedback จากนักวิเคราะห์, ชุดประเมินที่ใหญ่ขึ้น, การเขียน query ใหม่, และใช้ LLM-as-judge ประเมินคุณภาพรายงานโดยเทียบกับมนุษย์

การทำซ้ำผลลัพธ์

คำสั่ง: `python -m eval.run_retrieval_eval`, `python -m eval.run_e2e_eval --split test --round r3_outcome_flag_label_fallback`, `python -m eval.compare_rounds`; python 3.13.12; LSA seed 42; BM25 k1=1.5, b=0.75; embedding model sentence-transformers/all-MiniLM-L6-v2 ผลรายข้ออยู่ใน eval/results/per_example.csv และ eval/results/rounds/ การทดสอบ: `pytest` (offline, LLM จำลอง)

อ้างอิง: Nat-D/kmitl-llm-app-demo (แหล่งอ้างอิงของรายวิชา ใช้เป็นแนวคิดเท่านั้น); Robertson & Zaragoza (2009) BM25; Reimers & Gurevych (2019) Sentence-BERT; Deerwester et al. (1990) LSA; RFC 5737; ฟอนต์ LINE Seed Sans TH (SIL OFL 1.1)